

Step 3. Add Model Router and Gateway

As applications grow to involve more models, routers and gateways emerge to help you manage the complexity and costs of serving multiple models.

Router

Instead of using one model for all queries, you can have different solutions for different types of queries. This approach has several benefits. First, it allows specialized models, which can potentially perform better than a general-purpose model for specific queries. For example, you can have one model specialized in technical troubleshooting and another specialized in billing. Second, this can help you save costs. Instead of using one expensive model for all queries, you can route simpler queries to cheaper models.

A router typically consists of an *intent classifier* that predicts what the user is trying to do. Based on the predicted intent, the query is routed to the appropriate solution. As an example, consider different intentions relevant to a customer support chatbot:

- If the user wants to reset the password, route them to the FAQ page about recovering the password.
- If the request is to correct a billing mistake, route it to a human operator.
- If the request is about troubleshooting a technical issue, route it to a chatbot specialized in troubleshooting.

An intent classifier can prevent your system from engaging in out-of-scope conversations. If the query is deemed inappropriate, the chatbot can politely decline to respond using one of the stock responses without wasting an API call. For example, if the user asks who you would vote for in the upcoming election, a chatbot can respond with: “As a chatbot, I don’t have the ability to vote. If you have questions about our products, I’d be happy to help.”

An intent classifier can help the system detect ambiguous queries and ask for clarification. For example, in response to the query “Freezing”, the system might ask, “Do you want to freeze your account or are you talking about the weather?” or simply ask, “I’m sorry. Can you elaborate?”

Other routers can aid the model in deciding what to do next. For example, for an agent capable of multiple actions, a router can take the form of a *next-action predictor*: should the model use a code interpreter or a search API next? For a model with a memory system, a router can predict which part of the memory hierarchy the model should pull information from. Imagine that a user attaches a document that mentions Melbourne to the current conversation. Later on, the user asks: “What’s the cutest animal in Melbourne?” The model needs to decide whether to rely on the information in the attached document or to search the internet for this query.

Intent classifiers and next-action predictors can be implemented on top of foundation models. Many teams adapt smaller language models like GPT-2, BERT, and Llama 7B as their intent classifiers. Many teams opt to train even smaller classifiers from scratch. Routers should be fast and cheap so that they can use multiples of them without incurring significant extra latency and cost.

When routing queries to models with varying context limits, the query’s context might need to be adjusted accordingly. Consider a 1,000-token query that is slated for a model with a 4K context limit. The system then takes an action, e.g., a web search, that brings back 8,000-token context. You can either truncate the query’s context to fit the originally intended model or route the query to a model with a larger context limit.

Because routing is usually done by models, I put routing inside the Model API box in [Figure 10-5](#). Like scorers, routers are typically smaller than models used for generation.

Grouping routers together with other models makes models easier to manage. However, it’s important to note that routing often happens *before* retrieval. For example, before retrieval, a router can help determine if a query is in-scope and, if yes, if it needs retrieval. Routing can happen after retrieval, too, such as determining if a query should be routed to a human operator. However, routing - retrieval - generation - scoring is a much more common AI application pattern.

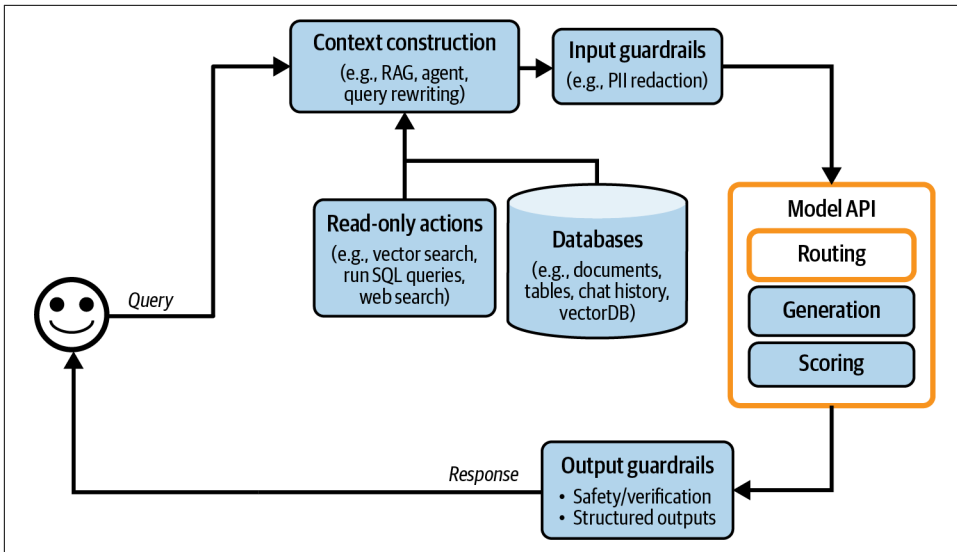


Figure 10-5. Routing helps the system use the optimal solution for each query.

Gateway

A model gateway is an intermediate layer that allows your organization to interface with different models in a unified and secure manner. The most basic functionality of a model gateway is to provide a unified interface to different models, including self-hosted models and models behind commercial APIs. A model gateway makes it easier to maintain your code. If a model API changes, you only need to update the gateway instead of updating all applications that depend on this API. Figure 10-6 shows a high-level visualization of a model gateway.

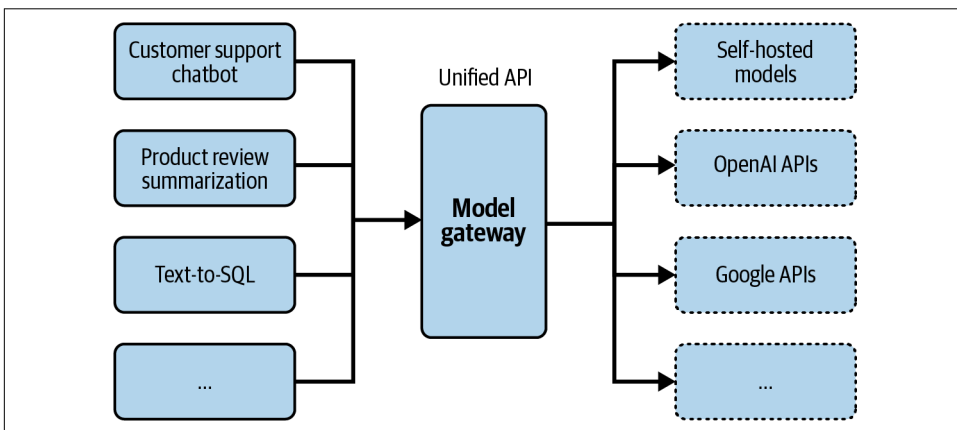


Figure 10-6. A model gateway provides a unified interface to work with different models.

In its simplest form, a model gateway is a unified wrapper. The following code example gives you an idea of how a model gateway might be implemented. It's not meant to be functional, as it doesn't contain any error checking or optimization:

```
import google.generativeai as genai
import openai

def openai_model(input_data, model_name, max_tokens):
    openai.api_key = os.environ["OPENAI_API_KEY"]
    response = openai.Completion.create(
        engine=model_name,
        prompt=input_data,
        max_tokens=max_tokens
    )
    return {"response": response.choices[0].text.strip()}

def gemini_model(input_data, model_name, max_tokens):
    genai.configure(api_key=os.environ["GOOGLE_API_KEY"])
    model = genai.GenerativeModel(model_name=model_name)
    response = model.generate_content(input_data, max_tokens=max_tokens)
    return {"response": response["choices"][0]["message"]["content"]}

@app.route('/model', methods=['POST'])
def model_gateway():
    data = request.get_json()
    model_type = data.get("model_type")
    model_name = data.get("model_name")
    input_data = data.get("input_data")
    max_tokens = data.get("max_tokens")

    if model_type == "openai":
        result = openai_model(input_data, model_name, max_tokens)
    elif model_type == "gemini":
        result = gemini_model(input_data, model_name, max_tokens)
    return jsonify(result)
```

A model gateway provides *access control and cost management*. Instead of giving everyone who wants access to the OpenAI API your organizational tokens, which can be easily leaked, you give people access only to the model gateway, creating a centralized and controlled point of access. The gateway can also implement fine-grained access controls, specifying which user or application should have access to which model. Moreover, the gateway can monitor and limit the usage of API calls, preventing abuse and managing costs effectively.

A model gateway can also be used to implement fallback policies to overcome rate limits or API failures (the latter is unfortunately common). When the primary API is unavailable, the gateway can route requests to alternative models, retry after a short wait, or handle failures gracefully in other ways. This ensures that your application can operate smoothly without interruptions.

Since requests and responses are already flowing through the gateway, it's a good place to implement other functionalities, such as load balancing, logging, and analytics. Some gateways even provide caching and guardrails.

Given that gateways are relatively straightforward to implement, there are many off-the-shelf gateways. Examples include [Portkey's AI Gateway](#), [MLflow AI Gateway](#), [Wealthsimple's LLM Gateway](#), [TrueFoundry](#), [Kong](#), and [Cloudflare](#).

In our architecture, the gateway now replaces the model API box, as shown in [Figure 10-7](#).

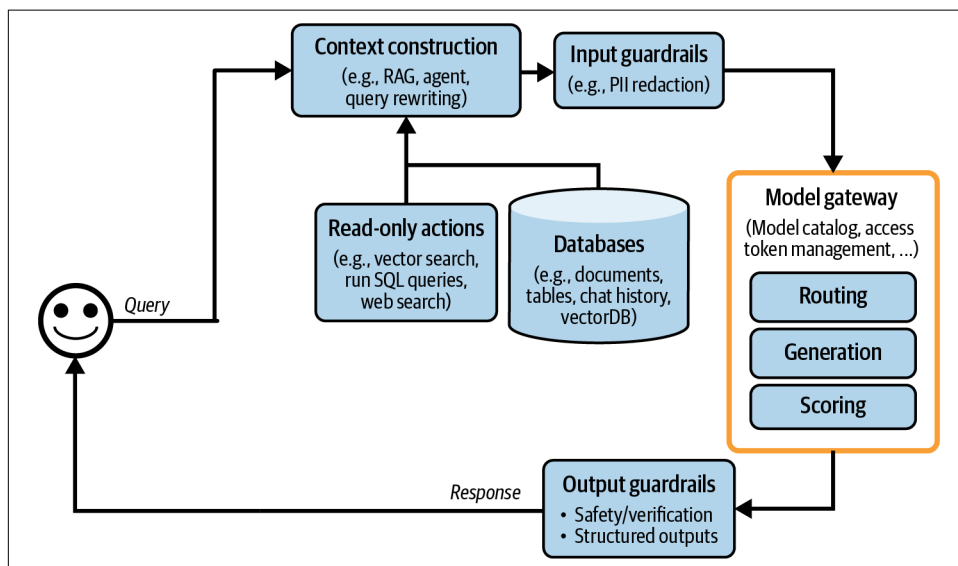


Figure 10-7. The architecture with the added routing and gateway modules.



A similar abstraction layer, such as a tool gateway, can also be useful for accessing a wide range of tools. It's not discussed in this book since it's not a common pattern as of this writing.

Step 4. Reduce Latency with Caches

Caching has long been integral to software applications to reduce latency and cost. Many ideas from software caching can be used for AI applications. Inference caching techniques, including KV caching and prompt caching, are discussed in [Chapter 9](#). This section focuses on system caching. Because caching is an old technology with a large amount of existing literature, this book will cover it only in broad strokes. In general, there are two major system caching mechanisms: exact caching and semantic caching.

Exact caching

With exact caching, cached items are used only when these exact items are requested. For example, if a user asks a model to summarize a product, the system checks the cache to see if a summary of this exact product exists. If yes, fetch this summary. If not, summarize the product and cache the summary.

Exact caching is also used for embedding-based retrieval to avoid redundant vector search. If an incoming query is already in the vector search cache, fetch the cached result. If not, perform a vector search for this query and cache the result.

Caching is especially appealing for queries that involve multiple steps (e.g., chain-of-thought) and/or time-consuming actions (e.g., retrieval, SQL execution, or web search).

An exact cache can be implemented using in-memory storage for fast retrieval. However, since in-memory storage is limited, a cache can also be implemented using databases like PostgreSQL, Redis, or tiered storage to balance speed and storage capacity. Having an eviction policy is crucial to manage the cache size and maintain performance. Common eviction policies include Least Recently Used (LRU), Least Frequently Used (LFU), and first in, first out (FIFO).

How long to keep a query in the cache depends on how likely this query is to be called again. User-specific queries, such as “What’s the status of my recent order?”, are less likely to be reused by other users and, therefore, shouldn’t be cached. Similarly, it makes less sense to cache time-sensitive queries such as “How’s the weather?” Many teams train a classifier to predict whether a query should be cached.



Caching, when not properly handled, can cause data leaks. Imagine you work for an ecommerce site, and user X asks a seemingly generic question such as: “What is the return policy for electronics products?” Because your return policy depends on the user’s membership, the system first retrieves user X’s information and then generates a response containing X’s information. Mistaking this query for a generic question, the system caches the answer. Later, when user Y asks the same question, the cached result is returned, revealing X’s information to Y.

Semantic caching

Unlike in exact caching, cached items are used even if they are only semantically similar, not identical, to the incoming query. Imagine one user asks, “What’s the capital of Vietnam?” and the model answers, “Hanoi”. Later, another user asks, “What’s the capital *city* of Vietnam?”, which is semantically the same question but with slightly different wording. With semantic caching, the system can reuse the answer from the first query instead of computing the new query from scratch. Reusing similar queries

increases the cache's hit rate and potentially reduces cost. However, semantic caching can reduce your model's performance.

Semantic caching works only if you have a reliable way of determining if two queries are similar. One common approach is to use semantic similarity, as discussed in [Chapter 3](#). As a refresh, semantic similarity works as follows:

1. For each query, generate its embedding using an embedding model.
2. Use vector search to find the cached embedding with the highest similar score to the current query embedding. Let's say this similarity score is X .
3. If X is higher than a certain similarity threshold, the cached query is considered similar, and the cached results are returned. If not, process this current query and cache it together with its embedding and results.

This approach requires a vector database to store the embeddings of cached queries.

Compared to other caching techniques, semantic caching's value is more dubious because many of its components are prone to failure. Its success relies on high-quality embeddings, functional vector search, and a reliable similarity metric. Setting the right similarity threshold can also be tricky, requiring a lot of trial and error. If the system mistakes the incoming query for one similar to another query, the returned response, fetched from the cache, will be incorrect.

In addition, semantic cache can be time-consuming and compute-intensive, as it involves a vector search. The speed and cost of this vector search depend on the size of your cached embeddings.

Semantic cache might still be worthwhile if the cache hit rate is high, meaning that a good portion of queries can be effectively answered by leveraging the cached results. However, before incorporating the complexities of a semantic cache, make sure to evaluate the associated efficiency, cost, and performance risks.

With the added cache systems, the platform looks like [Figure 10-8](#). A KV cache and prompt cache are typically implemented by model API providers, so they aren't shown in this image. To visualize them, I'd put them in the Model API box. There's a new arrow to add generated responses to the cache.

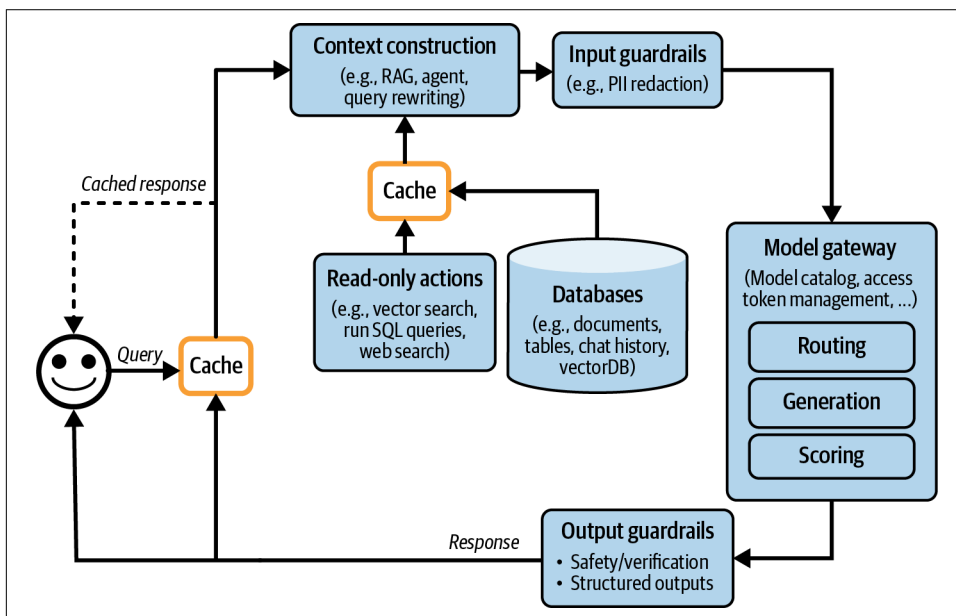


Figure 10-8. An AI application architecture with the added caches.

Step 5. Add Agent Patterns

The applications discussed so far are still fairly simple. Each query follows a sequential flow. However, as discussed in [Chapter 6](#), an application flow can be more complex with loops, parallel execution, and conditional branching. Agentic patterns, discussed in [Chapter 6](#), can help you build complex applications. For example, after the system generates an output, it might determine that it hasn't accomplished the task and that it needs to perform another retrieval to gather more information. The original response, together with the newly retrieved context, is passed into the same model or a different one. This creates a loop, as shown in [Figure 10-9](#).

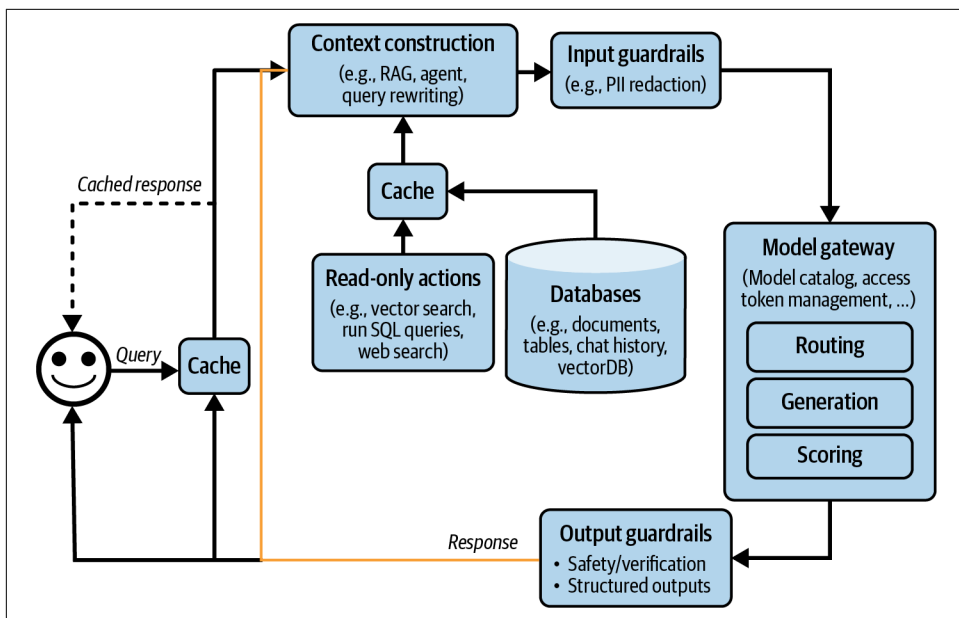


Figure 10-9. The yellow arrow allows the generated response to be fed back into the system, allowing more complex application patterns.

A model's outputs also can be used to invoke write actions, such as composing an email, placing an order, or initializing a bank transfer. Write actions allow a system to make changes to its environment directly. As discussed in [Chapter 6](#), write actions can make a system vastly more capable but also expose it to significantly more risks. Giving a model access to write actions should be done with the utmost care. With added write actions, the architecture looks like [Figure 10-10](#).

If you've followed all the steps so far, your architecture has likely grown quite complex. While complex systems can solve more tasks, they also introduce more failure modes, making them harder to debug due to the many potential points of failure. The next section will cover best practices for improving system observability.